

1 Problema Scuderia

1.1 Enunțul problemei

Presupunem că avem o matrice bidimensională cu n elemente organizate în linii de câte k elemente fiecare. Trebuie să găsim suma maximă posibilă a unei forme compuse din două benzi care se intersectează:

- O bandă orizontală de înălțime q (acoperă q linii consecutive)
- O bandă verticală de lățime p (acoperă p coloane consecutive)

Cele două benzi se suprapun într-o zonă dreptunghiulară, deci aria totală acoperită este $|H| + |V| - |H \cap V|$.

1.2 Observații cheie

1.2.1 Pentru ce avem nevoie de offset?

Problema menține o marjă de $OFFSET_MAX = 10^5 + 1$ pentru a permite accesarea indicilor negative în funcția $s(i, j)$, care returnează suma prefix a submatricei $(0, 0)$ până la (i, j) . Când $i < 0$ sau $j < 0$, funcția întoarce 0, simulând comportamentul unei matrice extinsă cu zerouri la stânga și deasupra.

Valoarea offset-ului se calculează ca $k - r$, unde r este un parametru care controlează această margine.

1.2.2 Tabelul de sume prefix bidimensionale

Soluția construiește o matrice de sume prefix în timp $O(n \cdot k)$:

$$pref[i][j] = a[i][j] + pref[i-1][j] + pref[i][j-1] - pref[i-1][j-1]$$

Cu această structură, suma oricărei submatrice poate fi calculată în $O(1)$:

$$suma(i_1, j_1, i_2, j_2) = pref[i_2][j_2] - pref[i_1-1][j_2] - pref[i_2][j_1-1] + pref[i_1-1][j_1-1]$$

Unde $i_1 \leq i_2$ și $j_1 \leq j_2$.

1.3 Soluția

1. Citim datele și calculăm suma totală a matricei.
2. Construim tabelul de sume prefix bidimensionale în-place în array-ul a .
3. Parcurgem toate pozițiile posibile pentru benzile orizontală și verticală:
 - Pentru fiecare linie de start i (începând de la q) și coloană j (de la $p-1$)
 - Calculăm separat suma benzii orizontale, a benzii verticale și a intersecției lor

- Suma totală pentru această configurație: $H + V - H \cap V$
4. Când coloana unei benzi depășește $k - 1$ (marginea din dreapta), tratăm un caz special cu decalaj.
 5. Reținem maximul tuturor sumelor calculate.

Formulele pentru sumele componentelor (linia i este linia curentă de start pentru banda orizontală):

$$\begin{aligned} H &= s(i, k - 1) - s(i - q, k - 1) \\ V &= s(\text{lastLine}, j) - s(\text{lastLine}, j - p) \\ H \cap V &= s(i, j) - s(i - q, j) + s(i - q, j - p) - s(i, j - p) \end{aligned}$$

Unde $s(x, y)$ este funcția care returnează suma prefix până la (x, y) .

1.4 Note de implementare

- Funcția $s(i, j)$ gestionează indicii negativi întorcând 0, ceea ce simplifică calculele de la margini.
- Array-ul a este dimensionat cu $2 \cdot \text{OFFSET_MAX}$ elemente suplimentare pentru a permite scrierea în siguranță a tabelului prefix.
- Cazul special când $p > k$ impune $p = k$, deoarece banda verticală nu poate fi mai lată decât matricea.
- Complexitatea totală este $O(n \cdot k)$ pentru preprocesare și $O(n \cdot k)$ pentru parcurgerea soluțiilor posibile.

1.5 Codul soluției

```
#include <fstream>
#include <iostream>
#include <math.h>
#include <limits.h>

using namespace std;

const int NMAX = 1e6 + 1;
const int OFFSET_MAX = 1e5 + 1;
int a[NMAX + 2 * OFFSET_MAX];
int n, k, p, q, r;

int s(int i, int j)
{
    if (i < 0 || j < 0)
        return 0;
```

```

        return a[i * k + j];
    }

    int getCoords(int x, int y)
    {
        return x * k + y;
    }

    int main()
    {
        ifstream fin("scuderia.in");
        ofstream fout("scuderia.out");

        fin >> n >> k >> p >> q >> r;

        if (p > k)
            p = k;

        int offset = k - r, totalSum = 0;

        for (int i = 0; i < n; ++i)
        {
            fin >> a[i + offset];
            totalSum += a[i + offset];
        }

        n += offset;
        int lastLine = n / k + (n % k != 0) - 1;

        // Construim tabelul de sume prefix
        for (int i = 0; i <= lastLine; ++i)
            for (int j = 0; j < k; ++j)
            {
                int ind = getCoords(i, j);
                if (i)
                    a[ind] += a[getCoords(i - 1, j)];
                if (j)
                    a[ind] += a[getCoords(i, j - 1)];

                if (i && j)
                    a[ind] -= a[getCoords(i - 1, j - 1)];
            }

        int maxSum = INT_MIN, colGap = k - p;

        for (int i = q; getCoords(i, k - 1) < n; ++i)
        {
            for (int j = p - 1; j < k; ++j)
            {
                int lineSum = s(i, k - 1) - s(i - q, k - 1),

```

```

        colSum = s(lastLine, j) - s(lastLine, j - p)
        ,
        intersectSum = s(i, j) - s(i - q, j) + s(i -
            q, j - p) - s(i, j - p);

    if (lineSum + colSum - intersectSum > maxSum)
    {
        maxSum = lineSum + colSum - intersectSum;
    }
}

for (int j = colGap; j < k - 1; ++j)
{
    int hSum = totalSum -
        (s(lastLine, j) - s(lastLine, j -
            colGap)) +
        (s(i, j) - s(i - q, j) - s(i, j -
            colGap) + s(i - q, j - colGap));

    if (hSum > maxSum)
    {
        maxSum = hSum;
    }
}

fout << maxSum << '\n';
return 0;
}

```

1.6 Concluzii

Problema demonstrează puterea tabelor de sume prefix pentru interogări $O(1)$ pe submatrice. Prin descompunerea formei complexe în componente simple (benzi orizontale, verticale și intersecția lor) și aplicarea principiului includerii-excluderii, putem evalua fiecare configurație posibilă în timp constant. Soluția finală are complexitate $O(n \cdot k)$ atât în spațiu cât și în timp, fiind eficientă pentru limitele date ($n \leq 10^6$, $k \leq 10^5$).